

Documentație tehnică — Adăugare client nou

Proiect: `chatbot-agency-template` | Stack: Node.js + TypeScript + Express + OpenAI

Cuprins

1. Arhitectura proiectului
 2. Structura unui client
 3. Pasul 1 — Creează folderul clientului
 4. Pasul 2 — config.json
 5. Pasul 3 — system-prompt.txt
 6. Pasul 4 — context.txt
 7. Cum procesează serverul fișierele
 8. API Reference
 9. Testare locală
 10. Checklist complet client nou
 11. Exemple pentru tipuri de business
-

1. Arhitectura proiectului

```
chatbot-agency-template/  
├─ clients/                ← un subfolder per client  
│   ├─ erikajurescu/  
│   ├─ bistro-luna/  
│   └─ example-client/  
├─ src/  
│   ├─ server.ts           ← Express API (endpoints)  
│   └─ core/  
│       ├─ clientConfig.ts ← citire config.json, listare clienți  
│       └─ chat.ts         ← construire system prompt + apel OpenAI  
├─ public/                 ← frontend static (UI chat)  
├─ .env                    ← OPENAI_API_KEY  
└─ package.json
```

Fluxul de date:

Frontend UI



POST /api/chat/:clientId { messages: [...] }



chat.ts → loadSystemPrompt(clientId)



- └─ citește config.json
- └─ citește prompts/system-prompt.txt
- └─ citește prompts/context.txt (opțional)

OpenAI API (gpt-4o-mini)



{ reply: "..." } → Frontend

2. Structura unui client

Fiecare client este un **subfolder** în `clients/` cu această structură:

```
clients/  
  [client-id]/  
    config.json          ← OBLIGATORIU  
    prompts/  
      system-prompt.txt ← OBLIGATORIU (comportament bot)  
      context.txt       ← opțional, dar recomandat (date business)
```

Regulă: Atât timp cât există `clients/<id>/config.json`, serverul îl detectează automat în `GET /api/clients`. Nu e nevoie să înregistrezi clientul nicăieri altundeva.

3. Pasul 1 — Creează folderul clientului

Alege un `clientId`: litere mici, cifre și `-`. Fără spații.

```
bash  
  
# din rădăcina proiectului:  
mkdir -p clients/numele-clientului/prompts
```

Sau copiază un client existent ca punct de start:

bash

```
cp -r clients/example-client clients/numele-clientului
```

4. Pasul 2 — config.json

Fișier: clients/[client-id]/config.json

```
json
{
  "id": "numele-clientului",
  "name": "Numele Afacerii SRL",
  "brandColor": "#4F46E5",
  "brandColors": {
    "primary": "#4F46E5",
    "secondary": "#8B5CF6",
    "assistantBubble": "#E5E7EB",
    "background": "#F4F5FB",
    "surface": "#FFFFFF"
  },
  "defaultLanguage": "ro",
  "description": "Scurtă descriere a afacerii pentru listing.",
  "toneOfVoice": "prietenoasă, clară, profesionistă",
  "openingMessage": "Bună! 🙋 Sunt asistentul virtual al [Nume]. Cu ce vă pot ajuta?",
  "allowedTopics": [
    "servicii",
    "programări",
    "tarife",
    "program",
    "contact"
  ]
}
```

Câmpuri detaliate

Câmp	Tip	Obligatoriu	Descriere
id	string	✓	Trebuie să fie identic cu numele folderului
name	string	✓	Apare în UI (header, dropdown)
brandColor	string	—	Hex color, fallback pentru compatibilitate

Câmp	Tip	Obligatoriu	Descriere
<code>brandColors</code>	object	—	Set complet culori UI (override <code>brandColor</code>)
<code>brandColors.primary</code>	string	—	Header, buton trimite
<code>brandColors.secondary</code>	string	—	Bubble mesaj user
<code>brandColors.assistantBubble</code>	string	—	Bubble mesaj bot
<code>brandColors.background</code>	string	—	Fundal pagină
<code>brandColors.surface</code>	string	—	Card/container chat
<code>defaultLanguage</code>	string	—	<code>"ro"</code> sau <code>"en"</code> . Default: <code>"ro"</code>
<code>description</code>	string	—	Apare în listing <code>/api/clients</code>
<code>toneOfVoice</code>	string	—	Injectat automat în system prompt de bază
<code>openingMessage</code>	string	—	Mesajul de bun-venit afișat de frontend
<code>allowedTopics</code>	array	—	Informativ; îl poți folosi în system-prompt

5. Pasul 3 — `system-prompt.txt`

Fișier: `clients/[client-id]/prompts/system-prompt.txt`

Acesta definește **comportamentul, regulile și limitările** botului. Se adaugă automat după prompt-ul de bază generat din `config.json`.

Template generic (business standard)

Ești asistentul virtual al [Nume Client], un [tip business] din România.

SCOP:

- răspunzi la întrebările clienților clar și politicos;
- oferi informații despre servicii, program, locație, tarife;
- ajuți utilizatorii să ajungă la o rezervare / programare când este relevant.

STIL:

- răspunzi în limba română;
- ton: [prietenoasă, clară, profesionistă];
- adresezi utilizatorul la persoana a II-a plural ("dumneavoastră");

- răspunsuri scurte: 1-3 paragrafe sau bullet points.

LIMITĂRI:

- dacă o informație lipsește, nu inventa; spune că nu ai detaliul și invită utilizatorul să contacteze direct businessul;
- dacă utilizatorul cere ceva complet în afara domeniului, explică politicos că nu este domeniul tău.

CONTACT:

Dacă nu știi răspunsul, îndrumă utilizatorul să contacteze direct echipa [Nume Client] la [telefon/email].

Template pentru programări (cu colectare date contact)

LOGICA PROGRAMĂRI:

Consideră că utilizatorul vrea o programare dacă mesajul conține:

- cuvinte ca: "programare", "aș vrea o ședință", "vreau să vin", "o sesiune", zile ale săptămânii, ore ("după 4", "dimineața" etc.)

În aceste situații:

1. Confirmă scurt că serviciul solicitat este disponibil.
2. NU cere tu zi/oră exactă dacă utilizatorul a menționat deja un interval.
3. Cere DIRECT datele de contact:
 - nume complet (sau minim prenume)
 - număr de telefon sau adresă de email
4. Spune că ora și data exacte se stabilesc ulterior direct cu echipa.

Template pentru tipuri sensibile (psiholog, clinică, cabinet medical)

LIMITĂRI SPECIALE:

- Nu pune diagnostice și nu recomanda medicamente.
- Nu oferi terapie/consultație completă în chat.
- Încurajează programarea față în față sau online.

SITUAȚII DE URGENȚĂ:

Dacă utilizatorul menționează gânduri de suicid, auto-vătămare sau violență:

- răspunde empatic;
- îndrumă imediat să sune la 112 sau să contacteze servicii medicale;
- explică că ești un asistent virtual și nu poți interveni direct.

6. Pasul 4 — context.txt

Fișier: `clients/[client-id]/prompts/context.txt`

Acest fișier este **opțional dar foarte recomandat**. Conține informații concrete despre afacere pe care botul le folosește ca sursă de adevăr. Nu se afișează utilizatorului — e doar pentru model.

Template

CONTEXT DESPRE [NUME CLIENT] - DOAR PENTRU MODEL

1. TIP AFACERE

[Descriere în 2-3 fraze: ce face, unde, pentru cine]

2. SERVICII / PRODUSE

- [Serviciu 1] - descriere scurtă, preț dacă există
- [Serviciu 2] - descriere scurtă
- [Serviciu 3] - descriere scurtă

Asistentul NU inventează alte servicii în afara acestei liste.

3. PROGRAM ȘI LOCAȚIE

- Adresă: [stradă, oraș]
- Program: L-V [ore], S [ore], D [ore / închis]
- Telefon: [număr]
- Email: [adresă]

4. TARIFE (dacă sunt publice)

- [Serviciu 1]: [preț] RON / EUR
- [Serviciu 2]: [preț] RON / EUR

Dacă nu există tarife în această listă, spune că tarifele se comunică la contactarea directă a echipei.

5. POLITICI

- Programări: [cum se fac – telefon, site, WhatsApp]
- Anulări: [termen minim de anulare]
- Plată: [cash, card, transfer]

6. ALTE INFORMAȚII UTILE

[Orice altceva relevant: parcare, acces, FAQ frecvente etc.]

7. Cum procesează serverul fișierele

Funcția `loadSystemPrompt()` din `src/core/chat.ts` construiește prompt-ul final astfel:

```
[PROMPT DE BAZĂ generat din config.json]
```

```
+ toneOfVoice  
+ defaultLanguage
```

```
[INSTRUCȚIUNI SUPLIMENTARE]
```

```
+ conținut din prompts/system-prompt.txt
```

```
[CONTEXT BUSINESS]
```

```
+ conținut din prompts/context.txt  
(cu instrucțiunea să nu fie copiat mot-a-mot)
```

Dacă `system-prompt.txt` sau `context.txt` lipsesc, serverul continuă fără eroare — nu strică nimic.

Modelul folosit: `gpt-4o-mini` (configurat în `chat.ts`).

8. API Reference

`GET /api/clients`

Returnează lista tuturor clienților detectați în `clients/`.

Response:

```
json  
  
[  
  {  
    "id": "erikajurescu",  
    "name": "Cabinet psihologic Erika Jurescu",  
    "brandColor": "#6A86D9",  
    "defaultLanguage": "ro",  
    "description": "..."  
  }  
]
```

`GET /api/clients/:clientId`

Returnează config-ul complet al unui client.

POST /api/chat/:clientId

Trimite un mesaj și primește răspunsul botului.

Body:

```
json
{
  "messages": [
    { "role": "user", "content": "Bună ziua!" },
    { "role": "assistant", "content": "Bună! Cu ce vă pot ajuta?" },
    { "role": "user", "content": "Vreau o programare vineri." }
  ]
}
```

Important: Frontend-ul trimite întregul istoric al conversației la fiecare mesaj nou. Serverul nu stochează stare — fiecare request este independent.

Response:

```
json
{
  "clientId": "erikajurescu",
  "clientName": "Cabinet psihologic Erika Jurescu",
  "reply": "Bineînțeles! Îmi puteți lăsa numele și un număr de telefon..."
}
```

Erori posibile:

Status	Mesaj	Cauză
400	<code>Field 'messages' is required</code>	Body gol sau lipsă câmp messages
404	<code>Client not found</code>	<code>clientId</code> inexistent în <code>clients/</code>
500	<code>OPENAI_API_KEY is not set</code>	Lipsă cheie în <code>.env</code>
500	<code>Unexpected error</code>	Eroare OpenAI sau altă eroare de server

9. Testare locală

```
bash
```



```
# 1. Porneste serverul
npm run dev

# 2. Verifica lista de clienti
curl http://localhost:3000/api/clients

# 3. Testeaza un chat
curl -X POST http://localhost:3000/api/chat/numele-clientului \
  -H "Content-Type: application/json" \
  -d '{"messages": [{"role": "user", "content": "Bună ziua!"}]}'
```

Sau deschide **http://localhost:3000** în browser și selectează clientul din dropdown.

Dacă clientul nu apare în dropdown:

- Verifică că `clients/[id]/config.json` există și e valid JSON
- Verifică că câmpul `"id"` din JSON corespunde cu numele folderului
- Repornește serverul (`npm run dev`)

Dacă răspunsurile nu sunt corecte:

- Ajustează `system-prompt.txt` (ton, reguli, limitări)
- Completează `context.txt` cu informații mai precise
- Verifică că nu există erori de sintaxă în fișierele `.txt`

10. Checklist complet client nou

- [] 1. Aleg clientId (ex: "restaurant-mario")
 - [] 2. Creez folderul: clients/restaurant-mario/prompts/
 - [] 3. Creez config.json cu id, name, brandColors, toneOfVoice, openingMessage
 - [] 4. Creez prompts/system-prompt.txt cu regulile de comportament
 - [] 5. Creez prompts/context.txt cu servicii, tarife, program, contact
 - [] 6. Pornesc serverul: npm run dev
 - [] 7. Verific că apare în dropdown la http://localhost:3000
 - [] 8. Testez minim 5 întrebări tipice ale clienților
 - [] 9. Ajustez prompturile dacă e necesar
 - [] 10. git add . && git commit -m "Add client: restaurant-mario" && git push
-

11. Exemple pentru tipuri de business

Restaurant / Cafenea

toneOfVoice: "caldă, primitoare, informală"

allowedTopics: ["meniu", "rezervări", "program", "locație", "parcare"]

system-prompt extras:

Ajuți clienții să afle despre meniu, să facă rezervări și să obțină indicații de localizare. Nu promite disponibilitate fără confirmare – spune că rezervarea se confirmă telefonic.

Salon / Clinică estetică

toneOfVoice: "elegantă, profesionistă, caldă"

allowedTopics: ["tratamente", "prețuri", "programări", "pregătire"]

system-prompt extras:

Prezintă tratamentele disponibile și prețurile (dacă există în context). Pentru programări, colectează: nume, telefon, serviciu dorit și preferință de zi/interval orar. Nu da sfaturi medicale despre proceduri.

Magazin online / E-commerce

toneOfVoice: "prietenoasă, utilă, concisă"

allowedTopics: ["produse", "livrare", "retur", "plată", "stoc"]

system-prompt extras:

Ajuți clienții să găsească produse, să înțeleagă politica de livrare și retur. Dacă nu știi stocul exact, trimite la pagina produsului sau la echipa de suport. Nu promite termene de livrare fără confirmare.

Cabinet medical / Stomatologie

toneOfVoice: "calmă, profesionistă, de încredere"

allowedTopics: ["servicii medicale", "programări", "tarife", "asigurări"]

system-prompt extras:

Nu pune diagnostice și nu recomanda tratamente. Prezintă serviciile disponibile și ajută la programare. Pentru urgențe medicale, îndrumă la camera de gardă sau 112.